Name : Muhammad Farhan
Roll no: 25K-0538
Section: BCS-3K
Course: Data Structures – Lab
Lab no: 08
Course Instructor: Miss Fatima Gado
Date of Submission: 08/11/2026

Task 1:

```
#include <iostream>
#include <string>

using namespace std;

// node structure for the bst
class Node
{
public:
    int data;
    Node* left;
    Node* right;

    Node(int val = 0) : data(val), left(nullptr), right(nullptr)
    {
    }
};

// binary search tree class
class BinaryTree
{
private:
    Node* root;
    int rejected_count;

    // helper function for inorder traversal
    void inorderHelper(Node* curr_node)
    {
        if (curr_node != nullptr)
        {
            inorderHelper(curr_node->left);
            cout << curr_node->data << " ";
            inorderHelper(curr_node->right);
        }
    }
```

```
// helper function for postorder traversal
void postorderHelper(Node* curr_node)
{
   if (curr_node != nullptr)
   {
      postorderHelper(curr_node->left);
      postorderHelper(curr_node->right);
      cout << curr_node->data << " ";
   }
}

// helper function to count nodes
int countNodesHelper(Node* curr_node)
{
   if (curr_node == nullptr)
   {
      return 0;
   }
   return 1 + countNodesHelper(curr_node->left) + countNodesHelper(curr_node->right);
}

// helper function to calculate tree height
int getHeightHelper(Node* curr_node)
{
   if (curr_node == nullptr)
   {
      return -1;
   }
   int left_height = getHeightHelper(curr_node->left);
   int right_height = getHeightHelper(curr_node->right);
   if (left_height > right_height)
   {
      return left_height + 1;
   }
   else
   {
      return right_height + 1;
   }
}

// helper function to delete all nodes
void destroyTree(Node* curr_node)
{
   if (curr_node != nullptr)
   {
      destroyTree(curr_node->left);
      destroyTree(curr_node->right);
      delete curr_node;
   }
}
```

```
public:
    BinaryTree(Node* root_node = nullptr) : root(root_node), rejected_count(0)
    {
    }

    // destructor
    ~BinaryTree()
    {
        destroyTree(root);
    }

    // insert initial codes without printing
    void insertInitial(int val)
    {
        insertNode(val, false);
    }

    // insert a code and report depth or duplicate
    void insertNode(int val, bool show_msg = true)
    {
        if (root == nullptr)
        {
            root = new Node(val);
            if (show_msg)
            {
                cout << val << " inserted at depth 0" << endl;
            }
            return;
        }

        Node* curr_node = root;
        int depth = 0;

        while (curr_node != nullptr)
        {
            if (val == curr_node->data)
            {
                if (show_msg)
                {
                    cout << val << " duplicate, rejected" << endl;
                    rejected_count++;
                }
                return;
            }

            depth++;
            if (val < curr_node->data)
            {
                if (curr_node->left == nullptr)
                {
                    curr_node->left = new Node(val);
                    if (show_msg)
```

```
            {
                cout << val << " inserted at depth " << depth << endl;
            }
            return;
        }
        curr_node = curr_node->left;
    }
    else
    {
        if (curr_node->right == nullptr)
        {
            curr_node->right = new Node(val);
            if (show_msg)
            {
                cout << val << " inserted at depth " << depth << endl;
            }
            return;
        }
        curr_node = curr_node->right;
    }
  }
}

// search for a code and print visited path
bool searchNode(int val)
{
  cout << "SEARCH " << val << ": path";
  Node* curr_node = root;
  bool found = false;

  while (curr_node != nullptr)
  {
    cout << " " << curr_node->data;
    if (curr_node->data == val)
    {
      found = true;
      break;
    }
    else if (val < curr_node->data)
    {
      curr_node = curr_node->left;
    }
    else
    {
      curr_node = curr_node->right;
    }
  }

  if (found)
  {
    cout << ", found" << endl;
  }
```

```
        else
        {
            cout << ", not found" << endl;
        }

        return found;
    }

    // display inorder traversal
    void printInorder()
    {
        cout << "Inorder: ";
        inorderHelper(root);
        cout << endl;
    }

    // display postorder traversal
    void printPostorder()
    {
        cout << "Postorder: ";
        postorderHelper(root);
        cout << endl;
    }

    // count total nodes in tree
    int countNodes()
   {
        return countNodesHelper(root);
    }

    // get tree height
    int getHeight()
    {
        return getHeightHelper(root);
    }

    // display final summary
    void displaySummary()
    {
        cout << "Nodes " << countNodes() << ", Height " << getHeight() << ", Rejected " <<
rejected_count << endl;
    }
};

int main()
{
    BinaryTree tree;

    // insert initial product codes
    tree.insertInitial(50);
    tree.insertInitial(30);
    tree.insertInitial(70);
```

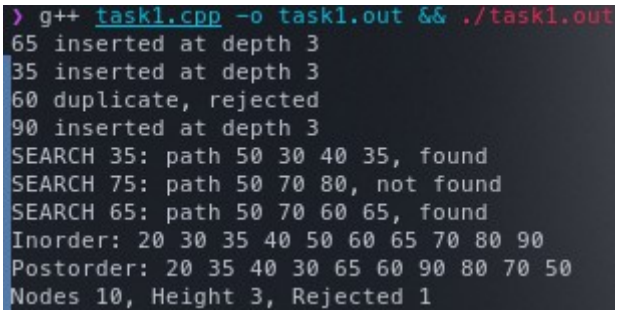
```
    tree.insertInitial(20);
    tree.insertInitial(40);
    tree.insertInitial(60);
    tree.insertInitial(80);

    // perform operations
    tree.insertNode(65);
    tree.insertNode(35);
    tree.insertNode(60);
    tree.insertNode(90);

    tree.searchNode(35);
    tree.searchNode(75);
    tree.searchNode(65);

    // display traversals and summary
    tree.printInorder();
    tree.printPostorder();
    tree.displaySummary();

    return 0;
}
```



Task 2:

```
#include <iostream>
#include <string>

using namespace std;

// node structure for bst
class Node
{
public:
    int data;
    Node* left;
    Node* right;

    Node(int val = 0) : data(val), left(nullptr), right(nullptr)
    {
    }
};

// binary search tree class
class BinaryTree
{
private:
    Node* root;
    int del_no_child;
    int del_one_child;
    int del_two_children;

    // helper for inserting a node
    Node* insertNodeHelper(Node* curr_node, int val, int depth, bool show_msg)
    {
        if (curr_node == nullptr)
        {
            if (show_msg)
            {
                cout << val << " inserted at depth " << depth << endl;
            }
            return new Node(val);
        }

        if (val < curr_node->data)
        {
            curr_node->left = insertNodeHelper(curr_node->left, val, depth + 1, show_msg);
        }
        else if (val > curr_node->data)
        {
            curr_node->right = insertNodeHelper(curr_node->right, val, depth + 1, show_msg);
        }
        return curr_node;
    }
```

```
// helper to find and remove minimum node
Node* extractMin(Node*& node)
{
   if (node->left == nullptr)
   {
      Node* min_node = node;
      node = node->right;
      return min_node;
   }
   return extractMin(node->left);
}

// helper to delete a node
Node* deleteNodeHelper(Node* curr_node, int val, int depth)
{
   if (curr_node == nullptr)
   {
      return nullptr;
   }

   if (val < curr_node->data)
   {
      curr_node->left = deleteNodeHelper(curr_node->left, val, depth + 1);
   }
   else if (val > curr_node->data)
   {
      curr_node->right = deleteNodeHelper(curr_node->right, val, depth + 1);
   }
   else
   {
      // case 1: 0 children
      if (curr_node->left == nullptr && curr_node->right == nullptr)
      {
         cout << curr_node->data << " depth " << depth << ", no child" << endl;
         del_no_child++;
         delete curr_node;
         return nullptr;
      }

      // case 2: 1 child
      if (curr_node->left == nullptr)
      {
         cout << curr_node->data << " depth " << depth << ", one child" << endl;
         del_one_child++;
         Node* temp_node = curr_node->right;
         delete curr_node;
         return temp_node;
      }
      if (curr_node->right == nullptr)
      {
         cout << curr_node->data << " depth " << depth << ", one child" << endl;
         del_one_child++;
```

```
            Node* temp_node = curr_node->left;
            delete curr_node;
            return temp_node;
        }

        // case 3: 2 children
        Node* successor = extractMin(curr_node->right);
        cout << curr_node->data << " depth " << depth << ", two children, replaced by " <<
successor->data << endl;
        del_two_children++;

        successor->left = curr_node->left;
        successor->right = curr_node->right;
        delete curr_node;
        return successor;
    }
    return curr_node;
}

// helper to find height
int getHeightHelper(Node* curr_node)
{
    if (curr_node == nullptr)
    {
        return -1;
    }
    int left_height = getHeightHelper(curr_node->left);
    int right_height = getHeightHelper(curr_node->right);
    if (left_height > right_height)
    {
        return left_height + 1;
    }
    else
    {
        return right_height + 1;
    }
}

// helper to count nodes at each level
void countLevelsHelper(Node* curr_node, int level, int level_counts[])
{
    if (curr_node == nullptr)
    {
        return;
    }
    level_counts[level]++;
    countLevelsHelper(curr_node->left, level + 1, level_counts);
    countLevelsHelper(curr_node->right, level + 1, level_counts);
}

// helper to clean up tree
void destroyTree(Node* curr_node)
```

```
    {
        if (curr_node != nullptr)
        {
            destroyTree(curr_node->left);
            destroyTree(curr_node->right);
            delete curr_node;
        }
    }

public:
    BinaryTree(Node* root_node = nullptr) : root(root_node), del_no_child(0), del_one_child(0),
del_two_children(0)
    {
    }

    // destructor
    ~BinaryTree()
    {
        destroyTree(root);
    }

    // insert initial codes without printing
    void insertInitial(int val)
    {
        root = insertNodeHelper(root, val, 0, false);
    }

    // insert a code and report depth
    void insertNode(int val)
    {
        root = insertNodeHelper(root, val, 0, true);
    }

    // delete a code and report details
    void deleteNode(int val)
    {
        root = deleteNodeHelper(root, val, 0);
    }

    // print level order traversal
    void printLevelOrder()
    {
        if (root == nullptr)
        {
            return;
        }

        Node* queue[100];
        int front = 0;
        int rear = 0;

        queue[rear++] = root;
```

```
        cout << "Level order:";
        while (front < rear)
        {
            Node* curr_node = queue[front++];
            cout << " " << curr_node->data;

            if (curr_node->left != nullptr)
            {
                queue[rear++] = curr_node->left;
            }
            if (curr_node->right != nullptr)
            {
                queue[rear++] = curr_node->right;
            }
        }
        cout << endl;
    }

    // print level stats and deletion counts
    void displayLevelStats()
    {
        int height = getHeightHelper(root);
        if (height == -1)
        {
            return;
        }

        int level_counts[100] = {0};
        countLevelsHelper(root, 0, level_counts);

        cout << endl;
        cout << "Level widths:";
        int max_nodes = -1;
        int widest_level = 0;

        for (int i = 0; i <= height; i++)
        {
            cout << " " << level_counts[i];
            if (level_counts[i] > max_nodes)
            {
                max_nodes = level_counts[i];
                widest_level = i;
            }
        }
        cout << endl;

        cout << "Widest level " << widest_level << endl;
        cout << "Height " << height << endl;
        cout << "Deletion kinds: no child " << del_no_child << ", one child " << del_one_child << ",
two children " << del_two_children << endl;
    }
```

```
};

int main()
{
    BinaryTree* tree = new BinaryTree();

    // insert initial product codes
    tree->insertInitial(40);
    tree->insertInitial(20);
    tree->insertInitial(60);
    tree->insertInitial(10);
    tree->insertInitial(30);
    tree->insertInitial(50);
    tree->insertInitial(70);
    tree->insertInitial(5);
    tree->insertInitial(15);
    tree->insertInitial(25);
    tree->insertInitial(35);

    // perform operations
    tree->insertNode(55);
    tree->deleteNode(20);
    tree->insertNode(12);
    tree->deleteNode(60);
    tree->deleteNode(5);

    // display level order and stats
    tree->printLevelOrder();
    tree->displayLevelStats();

    delete tree;
    return 0;
}
```

```
❯ g++ task2.cpp -o task2.out && ./task2.out
55 inserted at depth 3
20 depth 1, two children, replaced by 25
12 inserted at depth 4
60 depth 1, two children, replaced by 70
5 depth 3, no child
Level order: 40 25 70 10 30 50 15 35 55 12

Level widths: 1 2 3 3 1
Widest level 2
Height 4
Deletion kinds: no child 1, one child 0, two children 2
```

Task 3:

```
#include <iostream>
#include <string>

using namespace std;

// node structure for bst
class Node
{
public:
    int data;
    Node* left;
    Node* right;

    Node(int val = 0) : data(val), left(nullptr), right(nullptr)
    {
    }
};

// binary search tree class
class BinaryTree
{
private:
    Node* root;
    int del_no_child;
    int del_one_child;
    int del_two_children;

    // helper to insert a node
    Node* insertNodeHelper(Node* curr_node, int val)
    {
        if (curr_node == nullptr)
        {
            return new Node(val);
        }
        else if (val < curr_node->data)
        {
            curr_node->left = insertNodeHelper(curr_node->left, val);
        }
        else if (val > curr_node->data)
        {
            curr_node->right = insertNodeHelper(curr_node->right, val);
        }
        return curr_node;
    }

    // helper to extract maximum node from left subtree
    Node* extractMax(Node*& curr_node)
    {
        if (curr_node->right == nullptr)
        {
```

```
        Node* max_node = curr_node;
        curr_node = curr_node->left;
        return max_node;
    }
    return extractMax(curr_node->right);
}

// helper to delete a node
Node* deleteNodeHelper(Node* curr_node, int val, int depth)
{
    if (curr_node == nullptr)
    {
        return nullptr;
    }

    if (val < curr_node->data)
    {
        curr_node->left = deleteNodeHelper(curr_node->left, val, depth + 1);
    }
    else if (val > curr_node->data)
    {
        curr_node->right = deleteNodeHelper(curr_node->right, val, depth + 1);
    }
    else
    {
        // case 1: 0 children
        if (curr_node->left == nullptr && curr_node->right == nullptr)
        {
            cout << curr_node->data << " depth " << depth << ", no child" << endl;
            del_no_child++;
            delete curr_node;
            return nullptr;
        }

        // case 2: 1 child
        if (curr_node->left == nullptr)
        {
            cout << curr_node->data << " depth " << depth << ", one child" << endl;
            del_one_child++;
            Node* temp_node = curr_node->right;
            delete curr_node;
            return temp_node;
        }
        if (curr_node->right == nullptr)
        {
            cout << curr_node->data << " depth " << depth << ", one child" << endl;
            del_one_child++;
            Node* temp_node = curr_node->left;
            delete curr_node;
            return temp_node;
        }
```

```
            // case 3: 2 children
            Node* predecessor = extractMax(curr_node->left);
            cout << curr_node->data << " depth " << depth << ", two children, replaced by " <<
predecessor->data << " (next smaller)" << endl;
            del_two_children++;

            predecessor->left = curr_node->left;
            predecessor->right = curr_node->right;
            delete curr_node;
            return predecessor;
        }
        return curr_node;
    }

    // helper to calculate tree height
    int getHeightHelper(Node* curr_node)
    {
        if (curr_node == nullptr)
        {
            return -1;
        }
        int left_height = getHeightHelper(curr_node->left);
        int right_height = getHeightHelper(curr_node->right);
        if (left_height > right_height)
        {
            return left_height + 1;
        }
        else
        {
            return right_height + 1;
        }
    }

    // helper for range query
    void rangeHelper(Node* curr_node, int low, int high, int& count, int& sum)
    {
        if (curr_node == nullptr)
        {
            return;
        }

        if (curr_node->data > low)
        {
            rangeHelper(curr_node->left, low, high, count, sum);
        }

        if (curr_node->data >= low && curr_node->data <= high)
        {
            count++;
            sum += curr_node->data;
        }
```

```
        if (curr_node->data < high)
        {
            rangeHelper(curr_node->right, low, high, count, sum);
        }
    }

    // helper for preorder traversal
    void preorderHelper(Node* curr_node)
    {
        if (curr_node != nullptr)
        {
            cout << curr_node->data << " ";
            preorderHelper(curr_node->left);
            preorderHelper(curr_node->right);
        }
    }

    // helper for inorder traversal
    void inorderHelper(Node* curr_node)
    {
        if (curr_node != nullptr)
        {
            inorderHelper(curr_node->left);
            cout << curr_node->data << " ";
            inorderHelper(curr_node->right);
        }
    }

    // helper to clean up tree
    void destroyTree(Node* curr_node)
    {
        if (curr_node != nullptr)
        {
            destroyTree(curr_node->left);
            destroyTree(curr_node->right);
            delete curr_node;
        }
    }

public:
    BinaryTree(Node* root_node = nullptr) : root(root_node), del_no_child(0), del_one_child(0),
del_two_children(0)
    {
    }

    // destructor
    ~BinaryTree()
    {
        destroyTree(root);
    }

    // insert product code
```

```
void insertNode(int val)
{
   root = insertNodeHelper(root, val);
}

// delete a code and report details
void deleteNode(int val)
{
   root = deleteNodeHelper(root, val, 0);
}

// find floor value
void findFloor(int val)
{
   Node* curr_node = root;
   int floor_val = -1;
   bool found = false;

   while (curr_node != nullptr)
   {
      if (curr_node->data == val)
      {
         floor_val = curr_node->data;
         found = true;
         break;
      }
      else if (val < curr_node->data)
      {
         curr_node = curr_node->left;
      }
      else
      {
         floor_val = curr_node->data;
         found = true;
         curr_node = curr_node->right;
      }
   }

   cout << "FLOOR " << val << " = ";
   if (found)
   {
      cout << floor_val << endl;
   }
   else
   {
      cout << "none" << endl;
   }
}

// find ceil value
void findCeil(int val)
{
```

```
        Node* curr_node = root;
        int ceil_val = -1;
        bool found = false;

        while (curr_node != nullptr)
        {
            if (curr_node->data == val)
            {
                ceil_val = curr_node->data;
                found = true;
                break;
            }
            else if (val > curr_node->data)
            {
                curr_node = curr_node->right;
            }
            else
            {
                ceil_val = curr_node->data;
                found = true;
                curr_node = curr_node->left;
            }
        }

        cout << "CEIL " << val << " = ";
        if (found)
        {
            cout << ceil_val << endl;
        }
        else
        {
            cout << "none" << endl;
        }
    }

    // find count and sum in range
    void findRange(int low, int high)
    {
        int count = 0;
        int sum = 0;
        rangeHelper(root, low, high, count, sum);
        cout << "RANGE " << low << ".." << high << ": count " << count << ", sum " << sum <<
endl;
    }

    // print preorder traversal
    void printPreorder()
    {
        cout << "Preorder: ";
        preorderHelper(root);
        cout << endl;
    }
```

```
    // print inorder traversal
    void printInorder()
    {
        cout << "Inorder: ";
        inorderHelper(root);
        cout << endl;
    }

    // get tree height
    int getHeight()
    {
        return getHeightHelper(root);
    }

    // display summary stats
    void displaySummary()
    {
        cout << "Height " << getHeight() << endl;
        cout << "Deletion kinds: no child " << del_no_child  << ", one child " << del_one_child << ",
two children " << del_two_children << endl;
    }
};

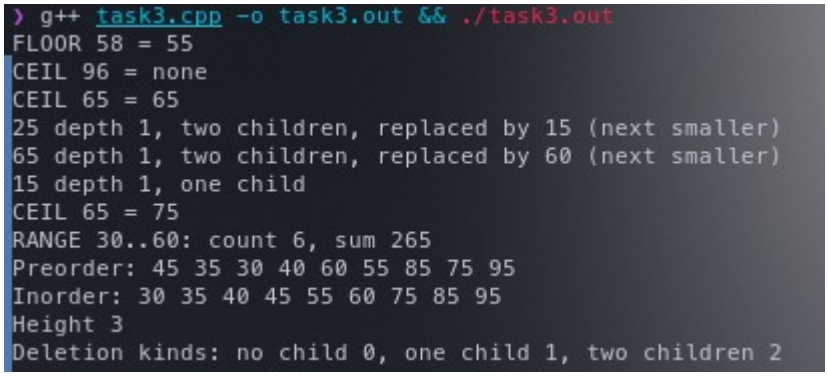
int main()
{
    BinaryTree* tree = new BinaryTree();

    // insert initial product codes
    tree->insertNode(45);
    tree->insertNode(25);
    tree->insertNode(65);
    tree->insertNode(15);
    tree->insertNode(35);
    tree->insertNode(55);
    tree->insertNode(85);
    tree->insertNode(30);
    tree->insertNode(40);
    tree->insertNode(60);
    tree->insertNode(75);
    tree->insertNode(95);

    // perform operations
    tree->findFloor(58);
    tree->findCeil(96);
    tree->findCeil(65);
    tree->deleteNode(25);
    tree->deleteNode(65);
    tree->deleteNode(15);
    tree->findCeil(65);
    tree->findRange(30, 60);
```

```
    // display traversals and summary
    tree->printPreorder();
    tree->printInorder();
    tree->displaySummary();

    delete tree;
    return 0;
}
```



Task 4:

```
#include <iostream>
#include <string>

using namespace std;

// node structure for bst
class Node
{
public:
    int data;
    Node* left;
    Node* right;

    Node(int val = 0) : data(val), left(nullptr), right(nullptr)
    {
    }
};

// binary search tree class
class BinaryTree
{
private:
    Node* root;
    int del_no_child;
    int del_one_child;
    int del_two_children;

    // helper for inserting a node
    Node* insertNodeHelper(Node* curr_node, int val, int depth, bool show_msg)
    {
        if (curr_node == nullptr)
        {
            if (show_msg)
            {
                cout << val << " inserted at depth " << depth << endl;
            }
            return new Node(val);
        }

        if (val < curr_node->data)
        {
            curr_node->left = insertNodeHelper(curr_node->left, val, depth + 1, show_msg);
        }
        else if (val > curr_node->data)
        {
            curr_node->right = insertNodeHelper(curr_node->right, val, depth + 1, show_msg);
        }
        return curr_node;
    }
```

```
// helper to search a value
Node* searchNodeHelper(Node* curr_node, int val)
{
   if (curr_node == nullptr)
   {
      return nullptr;
   }

   if (curr_node->data == val)
   {
      return curr_node;
   }
   else if (val < curr_node->data)
   {
      return searchNodeHelper(curr_node->left, val);
   }
   else
   {
      return searchNodeHelper(curr_node->right, val);
   }
}

// helper to find and remove minimum node
Node* extractMin(Node*& curr_node)
{
   if (curr_node->left == nullptr)
   {
      Node* min_node = curr_node;
      curr_node = curr_node->right;
      return min_node;
   }
   return extractMin(curr_node->left);
}

// helper to delete a node
Node* deleteNodeHelper(Node* curr_node, int val, int depth)
{
   if (curr_node == nullptr)
   {
      return nullptr;
   }

   if (val < curr_node->data)
   {
      curr_node->left = deleteNodeHelper(curr_node->left, val, depth + 1);
   }
   else if (val > curr_node->data)
   {
      curr_node->right = deleteNodeHelper(curr_node->right, val, depth + 1);
   }
   else
   {
```

```
            // case 1: 0 children
            if (curr_node->left == nullptr && curr_node->right == nullptr)
            {
                cout << curr_node->data << " depth " << depth << ", no child" << endl;
                del_no_child++;
                delete curr_node;
                return nullptr;
            }

            // case 2: 1 child
            if (curr_node->left == nullptr)
            {
                cout << curr_node->data << " depth " << depth << ", one child" << endl;
                del_one_child++;
                Node* temp_node = curr_node->right;
                delete curr_node;
                return temp_node;
            }
            if (curr_node->right == nullptr)
            {
                cout << curr_node->data << " depth " << depth << ", one child" << endl;
                del_one_child++;
                Node* temp_node = curr_node->left;
                delete curr_node;
                return temp_node;
            }

            // case 3: 2 children
            Node* successor = extractMin(curr_node->right);
            cout << curr_node->data << " depth " << depth << ", two children, replaced by " <<
successor->data << endl;
            del_two_children++;

            successor->left = curr_node->left;
            successor->right = curr_node->right;
            delete curr_node;
            return successor;
        }
        return curr_node;
    }

    // helper to find kth largest code
    void kthLargestHelper(Node* curr_node, int k, int& count, int& result)
    {
        if (curr_node == nullptr || count >= k)
        {
            return;
        }

        kthLargestHelper(curr_node->right, k, count, result);

        count++;
```

```
    if (count == k)
    {
        result = curr_node->data;
        return;
    }

    kthLargestHelper(curr_node->left, k, count, result);
}

// helper for postorder traversal
void postorderHelper(Node* curr_node)
{
    if (curr_node != nullptr)
    {
        postorderHelper(curr_node->left);
        postorderHelper(curr_node->right);
        cout << curr_node->data << " ";
    }
}

// helper to calculate tree height
int getHeightHelper(Node* curr_node)
{
    if (curr_node == nullptr)
    {
        return -1;
    }
    int left_height = getHeightHelper(curr_node->left);
    int right_height = getHeightHelper(curr_node->right);
    if (left_height > right_height)
    {
        return left_height + 1;
    }
    else
    {
        return right_height + 1;
    }
}

// helper to count total nodes
int countNodesHelper(Node* curr_node)
{
    if (curr_node == nullptr)
    {
        return 0;
    }
    return 1 + countNodesHelper(curr_node->left) + countNodesHelper(curr_node->right);
}

// helper to clean up tree
void destroyTree(Node* curr_node)
{
```

```
        if (curr_node != nullptr)
        {
            destroyTree(curr_node->left);
            destroyTree(curr_node->right);
            delete curr_node;
        }
    }

public:
    BinaryTree(Node* root_node = nullptr) : root(root_node), del_no_child(0), del_one_child(0),
del_two_children(0)
    {
    }

    // destructor
    ~BinaryTree()
    {
        destroyTree(root);
    }

    // insert initial codes without printing
    void insertInitial(int val)
    {
        root = insertNodeHelper(root, val, 0, false);
    }

    // insert a code and report depth
    void insertNode(int val)
    {
        root = insertNodeHelper(root, val, 0, true);
    }

    // find lowest common ancestor
    void findLca(int val1, int val2)
    {
        if (searchNodeHelper(root, val1) == nullptr || searchNodeHelper(root, val2) == nullptr)
        {
            cout << "LCA " << val1 << " " << val2 << ": LCA not possible" << endl;
            return;
        }

        Node* curr_node = root;
        while (curr_node != nullptr)
        {
            if (val1 < curr_node->data && val2 < curr_node->data)
            {
                curr_node = curr_node->left;
            }
            else if (val1 > curr_node->data && val2 > curr_node->data)
            {
                curr_node = curr_node->right;
            }
```

```
        else
        {
            cout << "LCA " << val1 << " " << val2 << " = " << curr_node->data << endl;
            return;
        }
    }
}

// find kth largest code
void findKthLargest(int k)
{
    int count = 0;
    int result = -1;
    kthLargestHelper(root, k, count, result);
    cout << "KTH " << k << " = " << result << endl;
}

// delete a code and report details
void deleteNode(int val)
{
    root = deleteNodeHelper(root, val, 0);
}

// print postorder traversal
void printPostorder()
{
    cout << "Postorder: ";
    postorderHelper(root);
    cout << endl;
}

// get tree height
int getHeight()
{
    return getHeightHelper(root);
}

// count total nodes
int countNodes()
{
    return countNodesHelper(root);
}

// display summary stats
void displaySummary()
{
    cout << "Height " << getHeight() << endl;
    cout << "Nodes " << countNodes() << endl;
    cout << "Deletion kinds: no child " << del_no_child
         << ", one child " << del_one_child
         << ", two children " << del_two_children << endl;
}
```

```
};

int main()
{
    BinaryTree* tree = new BinaryTree();

    // insert initial product codes
    tree->insertInitial(50);
    tree->insertInitial(25);
    tree->insertInitial(75);
    tree->insertInitial(10);
    tree->insertInitial(35);
    tree->insertInitial(60);
    tree->insertInitial(90);
    tree->insertInitial(5);
    tree->insertInitial(30);
    tree->insertInitial(40);
    tree->insertInitial(55);
    tree->insertInitial(65);

    // perform operations
    tree->findLca(30, 40);
    tree->findLca(5, 40);
    tree->findLca(55, 90);
    tree->findLca(30, 99);

    tree->insertNode(62);
    tree->deleteNode(75);
    tree->findKthLargest(3);
    tree->deleteNode(25);
    tree->findKthLargest(8);
    tree->deleteNode(5);
    tree->deleteNode(35);

    // display traversals and summary
    tree->printPostorder();
    tree->displaySummary();

    delete tree;
    return 0;
}
```

